ArcadeDB: One ACID Engine for Documents, Graphs, Vectors, and Time Series

[TODO: working title — alternatives in .notes strategy doc]

Taewoon Kim
HumemAI
taewoon@humem.ai

Roberto Franchini
ArcadeData
[TODO: email]

Christian Himpe
[TODO: affiliation]
[TODO: email]

Luca Garulli
ArcadeData
[TODO: email]

***Abstract*—[TODO: Written last. Spine: multi-model unification as a real, adopted system — one page/WAL/MVCC-transaction pipeline under documents, graphs, key-value, time series, and vectors; every index type (LSM, full-text, geo, hash, dense/sparse vector) commits inside the same transaction; Raft replication ships model-agnostic WAL page diffs; one jar runs embedded in-process or as a server speaking six wire protocols. Deep dive: what adding the vector model to a unified substrate takes, including the deliberate transactional-source-of-truth / eventually-consistent-ANN-graph tradeoff. Evaluation vs specialist engines + lessons from a decade of multi-model engineering (OrientDB lineage).]**

***Index Terms*—multi-model databases, ACID transactions, vector search, graph databases, embedded databases**

I. Introduction

[TODO: Vector-era motivation: apps hold documents + relationships + embeddings of the same entities; composed specialist stacks pay consistency/ops glue. Thesis: unification at the storage layer, not the API gateway. Claims 1–4 from the audit. Explicitly claim integration, not new algorithms.]

II. Background and Design Goals

[TODO: OrientDB lineage (”second system”); design goals; what industry usage demanded. Co-author input needed here. Third-party academic adoption: DataAasee [1], a metadata catalog for research libraries built on ArcadeDB (multi-model documents+graph use case); co-author ”why we chose it” paragraph goes here or in Lessons.]

III. Architecture: One Substrate, Five Models

[TODO: Page -¿ WAL -¿ optimistic-MVCC transaction pipeline; PaginatedComponent inventory (buckets, LSM trees, vector segments, time-series buckets); index changes committed in commit1stPhase; RID-linked adjacency + LightEdges (honest framing); time-series engine paragraph.]

IV. Case Study: Adding the Vector Model

[TODO: Dense (JVector HNSW) + LSM_SPARSE_VECTOR (BlockMax-WAND DAAT, RID-delta compression, quantization, size-tiered compaction). The tradeoff, stated plainly: vector records are transactional/WAL'd/Raft-replicated; the ANN graph is an asynchronously maintained, rebuildable derived structure. Include the measurement-integrity story (retracted early number) in a sidebar-style paragraph if space allows.]

V. Query and Deployment Surfaces

[TODO: One executor + result model behind SQL/Cypher/GraphQL/Mongo (Gremlin via TinkerPop over the same storage); embedded in-process vs server; six wire protocols; cite the SciPy paper for Python binding internals.]

VI. High Availability

[TODO: Ratis-based Raft shipping shared-WAL page diffs =¿ model-agnostic replication; quorum/membership hardening lessons.]

VII. Evaluation

[TODO: E1 sparse DB-vs-DB (Qdrant, Milvus) at 100k/1M/10M; E2 unified-ACID hybrid transaction vs composed stack incl. atomicity demonstration; E3 durability (kill -9) + Raft failover; E4 embedded vs server deployment axis; E5 dense sanity. Protocol: docker, pinned digests, identical cpuset/memory caps, N=5 mean±std, serial re-runs for all reported numbers.]

VIII. Lessons Learned and Tradeoffs

[TODO: The cost of unification (measured); where specialists win; single-node ceiling; async-ANN consequences; production war stories (co-authors).]

IX. Related Work

[TODO: Multi-model (Cosmos DB API-level vs our storage-level; ArangoDB; Lu & Holubová survey; UniBench/M2Bench), vector DBs (Milvus, Manu, Qdrant, pgvector), sparse retrieval (SPLADE, BlockMax-WAND), embedded engines (SQLite, DuckDB).]

X. Conclusion

[TODO:]

Acknowledgment

[TODO: AI-generated-content disclosure REQUIRED by ICDE 2027: name tools and affected content areas (drafting/editing assistance, experiment tooling). Keep updated as the paper evolves.]

REFERENCES

[1] C. Himpe, “DatAasee – a metadata-lake as metadata catalog for a virtual data-lake,” *arXiv preprint arXiv:2409.05512*, 2024.